МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет компьютерных систем и сетей
Кафедра программного обеспечения информационных технологий

ОТЧЕТ
по лабораторной работе № 2
по дисциплине «Программирование»

Тема: «Изучение механизмов перегрузки операций в языке C++»

Выполнил:
студент гр. 550503
Минкевич А. С.

Проверил:
ст. преподаватель
Ковальчук А. М.

Минск, 2026

ВВЕДЕНИЕ

0.0.1 Цель работы: изучить механизмы перегрузки операций в языке C++, получить практические навыки проектирования пользовательских классов, создания конструкторов (в том числе копирования) и перегрузки унарных, бинарных операций, а также операций ввода-вывода.

Задание на лабораторную работу: Реализовать класс `String` для работы со строками символов. Перегрузить следующие операции:

– Арифметические: `+`, `-`, `+=`.

– Присваивание: `=`.

– Инкремент и декремент (префиксные и постфиксные): `++`, `--`.

– Операции сравнения: `<`, `>`, `>=`, `<=`, `==`, `!=`.

– Доступ по индексу: `[]`.

– Выделение подстроки: `()(int, int)`.

– Ввод/вывод: `<<`, `>>`.

Необходимо реализовать конструктор без параметров, конструктор с параметрами и конструктор копирования. Предусмотреть ограничение на ввод текста (80 символов) для корректного отображения на экране.

1 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

1.1 Перегрузка операций и спецификатор friend

1.1.1 В языке C++ перегрузка операций может быть реализована как методом класса, так и внешней (глобальной) функцией.

Использование внешней функции, объявленной со спецификатором `friend`, строго обязательно для операций ввода-вывода (<<, >>), так как левым операндом в них выступает объект стандартного потока (`std::ostream` или `std::istream`), а не объект пользовательского класса.

Также `friend` необходим для бинарных операций, у которых левым операндом является не объект класса. Например, при конкатенации строки-литерала и объекта класса: `MyString d = "BSUIR" + a;` Если бы оператор `+` был методом класса, левым операндом обязан был бы быть объект `MyString`, и данное выражение вызвало бы ошибку компиляции. Поэтому в классе объявлена отдельная внешняя перегрузка `friend operator+(const char*, const MyString&)`, имеющая доступ к закрытым полям класса.

1.2 Использование спецификатора explicit

1.2.1 Спецификатор `explicit` используется перед объявлением конструкторов, чтобы запретить компилятору выполнять неявные преобразования типов. Это позволяет избежать трудноуловимых логических ошибок.

В разработанном классе `explicit` применён к конструктору преобразования `MyString(const char* str)`. Благодаря этому строковый литерал не превращается в объект класса автоматически, и компилятор не допускает случайных неявных преобразований. Чтобы выражения вида `"Prefix_" + A` и `B + "_Suffix"` при этом оставались допустимыми, для них явно определены дополнительные перегрузки оператора `+` с параметром `const char*`.

1.3 Перегрузка префиксных и постфиксных операций

1.3.1 Префиксные и постфиксные формы операций `++` и `--` различаются сигнатурой: постфиксная форма принимает фиктивный параметр типа `int`. Префиксная операция изменяет объект и возвращает ссылку на него, постфиксная возвращает копию объекта, сохранённую до изменения.

2 ХОД РАБОТЫ

2.1 Проектирование класса MyString

2.1.1 Разработан класс `MyString`, содержащий закрытые поля `data` (указатель на динамический массив символов), `length` (длина строки) и статическую константу `MAX_INPUT = 80`, ограничивающую размер буфера при вводе с клавиатуры.

Реализованы:

– Конструктор по умолчанию (создаёт пустую строку, в массиве хранится только символ `'\0'`).

– Конструктор преобразования из `const char*` со спецификатором `explicit` (вычисляет длину и выделяет память с запасом под завершающий нуль).

– Конструктор копирования (выделяет новую память и копирует содержимое, предотвращая ошибку двойного освобождения памяти).

– Деструктор, освобождающий память с помощью `delete[]`.

– Оператор присваивания с проверкой на самоприсваивание.

2.2 Реализация перегруженных операторов

2.2.1 Для реализации требуемого функционала были перегружены следующие операции:

1 `operator+` реализован как `friend` функция в трёх вариантах: `MyString + MyString`, `const char* + MyString` и `MyString + const char*`. Это позволяет формировать цепочки вида `"HELLO_" + A + B`.

2 `operator-` реализован как `friend` функция. Она ищет в левой строке первое вхождение правой строки с помощью `std::strstr` и удаляет его. Если подстрока не найдена или пуста, возвращается копия левой строки.

3 `operator+=` реализован как метод класса, так как он изменяет сам объект (левый операнд): выделяется новый массив нужного размера, в него копируются обе строки, старый массив освобождается.

4 `operator++` и `operator--` (префиксные и постфиксные) реализованы как методы класса. Они увеличивают или уменьшают код каждого символа строки на единицу. Префиксные формы возвращают ссылку на объект, постфиксные — его копию до изменения.

5 `operator[]` возвращает ссылку на символ по индексу. При выходе индекса за границы строки выбрасывается исключение `std::out_of_range`.

6 `operator()(int start, int len)` возвращает новую строку длиной не более `len`, скопированную из исходной, начиная с позиции `start`. Если запрошенный диапазон выходит за границы, длина обрезается, а при некорректных параметрах возвращается пустая строка.

7 Операции сравнения `<`, `>`, `<=`, `>=` реализованы как `friend` функции с использованием `std::strcmp` (лексикографический порядок).

8 `operator<<` и `operator>>` реализованы как `friend` функции. При вводе используется `getline` с буфером ограниченного размера, что защищает от переполнения.

2.3 Организация функции main

2.3.1 Для проверки операций разработано интерактивное меню. В нём можно ввести новые строки A, B, C, а также по отдельности проверить сложение, вычитание, составное присваивание, инкремент и декремент, сравнения, доступ по индексу и выделение подстроки, а также сложное выражение `D = "HELLO_" + A + B - C`, объединяющее несколько перегруженных операций.

3 РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ

3.0.1 Программа была собрана и протестирована с помощью инструмента AddressSanitizer: ошибок работы с памятью и утечек не обнаружено. Ниже представлен результат вывода программы для начальных значений строк `A = BSUIR`, `B = FKSIS`, `C = KS` (повторяющееся меню опущено).

Пример вывода консоли при выполнении программы

```
Select operation: 2
A + B = BSUIRFKSIS
"Prefix_" + A = Prefix_BSUIR
B + "_Suffix" = FKSIS_Suffix

Select operation: 3
A - B = BSUIR
A - C = BSUIR

Select operation: 5
Original A = BSUIR
After A++ = CTVJS
After --A = BSUIR

Select operation: 6
Comparing A and B:
A is LESS than B
A is LESS OR EQUAL to B

Select operation: 7
Enter index to view A[i]: 2
A[2] = U
Enter start and length for Substring A(start, len): 1 3
A(1, 3) = SUI

Select operation: 7
Enter index to view A[i]: 10
A[10] = Exception: Index out of bounds!
Enter start and length for Substring A(start, len): 0 0
A(0, 0) =

Select operation: 8
Executing: D = "HELLO_" + A + B - C
Result D = HELLO_BSUIRFIS
```

ЗАКЛЮЧЕНИЕ

3.0.2 В ходе выполнения лабораторной работы были изучены принципы перегрузки операций в языке C++. Был спроектирован и реализован пользовательский класс `MyString`, инкапсулирующий работу с динамическими массивами символов.

Практически закреплены навыки работы со спецификаторами `friend` и `explicit`, изучена разница между реализацией перегрузки через методы класса и внешние функции. Перегрузка операторов `++` и `--` позволила реализовать нестандартный функционал (сдвиг кодов символов строки). Реализовано безопасное чтение из потока с ограничением длины строки и проверка выхода за границы строки при доступе по индексу, а также предотвращены утечки памяти с помощью деструктора, конструктора копирования и корректной реализации оператора присваивания.